GovSmart: A Web-Based Citizen Complaint Management System Using React, FastAPI, and Role-Based Access Control

Group 6
Department of Informatics Engineering
Politeknik Elektronika Negeri Surabaya (PENS)
Surabaya, Indonesia
{kelompok6}@student.pens.ac.id

***Abstract*—Transparent and accountable public complaint management is a cornerstone of good governance. This paper presents GovSmart, a full-stack web application for managing citizen complaints directed at local government agencies. The system adopts a modern decoupled architecture, with a React.js single-page application on the frontend and a FastAPI (Python) REST API on the backend, backed by a PostgreSQL relational database. GovSmart implements a four-tier *Role-Based Access Control* (RBAC) scheme—covering *super_admin*, *admin*, *admin_dinas* (department administrator), and *warga* (citizen)—combined with an AI-assisted complaint routing engine that automatically dispatches incoming reports to the appropriate government department. Key engineering contributions include: (1) a secure Blob-based document export mechanism for PDF and CSV generation that correctly forwards JWT Bearer tokens; (2) a hierarchical role extension for export endpoints that accommodates super-admin privileges; and (3) a citizen-facing feedback document viewer enabling full transparency of governmental responses. System evaluation through 38 unit and integration tests confirms the correctness of RBAC enforcement, routing accuracy, and export functionality across all roles.**

***Index Terms*—citizen complaint management, e-government, FastAPI, React, role-based access control, AI routing, JWT authentication, document export**

I. Introduction

The rapid advancement of digital technology has created an expectation among citizens that governmental services be accessible, transparent, and responsive [1]. In Indonesia, public complaints about infrastructure, sanitation, social welfare, and other civic matters are frequently handled through informal channels—telephone calls, paper forms, or in-person visits—leading to inconsistent service quality and a lack of audit trails [2].

Electronic government (e-government) initiatives have demonstrated that web-based complaint management platforms can significantly reduce response times and improve citizen satisfaction [3]. However, many existing implementations lack fine-grained access control, automated routing to the correct department, and secure document distribution mechanisms, all of which are prerequisites for production deployment in a multi-department government environment.

This paper introduces **GovSmart**, a production-grade citizen complaint management system that addresses the above limitations. The system was developed iteratively and encompasses the following contributions:

- A full-stack decoupled architecture separating the React.js frontend from a FastAPI backend via a versioned REST API.
- A four-role RBAC scheme enforced at both API and UI layers.
- An AI-assisted complaint routing engine that classifies incoming reports by department and urgency level.
- A Blob-based, token-authenticated document export subsystem supporting CSV and printable HTML/PDF output.
- A citizen-facing feedback document viewer for official governmental responses.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system design. Section IV describes key implementation details. Section V evaluates the system, and Section VII concludes the paper.

II. Related Work

A. E-Government Complaint Systems

Several studies have examined online complaint management in the public sector. Janowski [4] identified four stages of digital government evolution: digitization, transformation, engagement, and contextualization. Complaint systems that offer real-time tracking and authenticated reporting reside primarily in the *engagement* phase. Dwivedi et al. [5] found that transparency of response status is the strongest predictor of citizen satisfaction in digital public services.

B. Role-Based Access Control in Web Applications

RBAC, standardized in ANSI INCITS 359-2004, restricts system access based on organizational roles rather than individual identities [6]. In multi-department government systems, hierarchical RBAC (hRBAC) is particularly relevant because

it allows a super-administrator to inherit permissions of subordinate roles while maintaining separation of duties [7].

C. JWT Authentication and Secure File Download

JSON Web Tokens (JWT) are the de facto standard for stateless authentication in RESTful services [8]. A common pitfall when combining token authentication with file downloads is that browser-initiated navigation (`window.open()`) cannot carry custom HTTP headers, causing 401/403 errors. The Blob Object URL pattern—fetching the resource with `fetch()` and constructing a temporary object URL—resolves this issue without exposing tokens in query strings [9].

D. AI-Assisted Text Classification for Routing

Automatic complaint routing maps free-text descriptions to predefined organizational categories. Keyword-based rule engines offer interpretability and low latency, making them suitable for resource-constrained government servers [10]. GovSmart employs a hybrid approach combining keyword normalization with confidence scoring.

III. System Design

A. Architecture Overview

GovSmart follows a three-tier decoupled architecture:

1) **Presentation Tier:** React.js SPA built with Vite. All API calls use `fetch()` with Bearer token injection.
2) **Application Tier:** FastAPI application containerized with Docker, exposing a versioned REST API under `/api/v1/`. SQLAlchemy 2.0 ORM manages database interactions.
3) **Data Tier:** PostgreSQL database with tables for *users*, *departments*, *reports*, and *audit_logs*.

B. Database Schema

The core relational schema consists of four tables:

- `users`: stores credentials, role, and optional `dept_id` foreign key.
- `departments`: registry of government agencies.
- `reports`: complaint records including title, body, urgency, status, GPS coordinates, evidence photo (`foto_bukti`), and feedback document (`foto_feedback`).
- `audit_logs`: immutable history of every status change with actor, timestamp, and notes.

All models use SQLAlchemy 2.0 `Mapped[T] = mapped_column(...)` syntax to guarantee static type safety across the codebase.

C. Role-Based Access Control

Table I summarizes the four roles and their primary permissions.

TABLE I
RBAC Role Permissions in GovSmart

Role	Key Permissions
`warga`	Submit, edit (pre-processing only), cancel complaints; view own reports; view feedback documents
`admin_dinas`	Update status; upload feedback files; export own-department reports
`admin`	All `admin_dinas` permissions across all departments; manage users
`super_admin`	All `admin` permissions; register new `admin_dinas` accounts; cross-department export

D. AI Complaint Routing Engine

When a citizen submits a complaint, the routing engine in `core/ai_routing.py` executes the following pipeline:

1) **Normalization:** Lowercase conversion and typo correction via a curated substitution dictionary (e.g., *jln* → *jalan*).
2) **Keyword Matching:** Each registered department maintains a keyword list; matching keywords are counted per department.
3) **Confidence Scoring:** A department is selected when its count exceeds a configurable threshold; ties are broken by keyword specificity weight.
4) **Urgency Classification:** Separate keyword sets classify urgency as `darurat` (emergency), `penting` (important), or `normal`.

Unrouted reports can be retroactively routed via the `POST /api/v1/reports/sync-routing` admin-only endpoint.

IV. Implementation

A. Backend: FastAPI and SQLAlchemy

The backend is structured as a standard FastAPI project:

Listing 1. Backend module structure

```
backend/app/
  api/endpoints/   # reports.py, users.py, chat.py
  core/            # ai_routing.py, upload.py, security.py
  crud/            # report.py, department.py
  models/          # report.py, user.py, department.py
  schemas/         # report.py, user.py
  database.py      # engine, session, auto-migration
```

Every endpoint handler uses FastAPI's dependency injection: `current_user: User = Depends(get_current_user)` for authentication and `db: Session = Depends(get_db)` for database access.

B. Secure Document Export via Blob Object URL

A recurring security issue in web applications is the inability to pass HTTP Authorization headers when triggering file downloads via browser navigation. GovSmart resolves this using the *Blob Object URL* pattern shown in Listing 2.

Listing 2. Blob-based authenticated PDF export

```
const handleExportPDF = async () => {
  const res = await fetch(
    `/api/v1/reports/export/html?${params}`,
    { headers: {
        Authorization: `Bearer ${token}` } }
  );
  if (!res.ok) throw new Error("Export failed");
  const html = await res.text();
  const blob = new Blob(
    [html], { type: "text/html;charset=utf-8" }
  );
  const url = URL.createObjectURL(blob);
  const win = window.open(url, "_blank");
  if (win)
    setTimeout(
      () => URL.revokeObjectURL(url), 10000
    );
};
```

The backend `/export/html` endpoint generates a print-ready HTML document styled for A4 output; the user invokes the browser's built-in print dialog (`Ctrl+P`/`Cmd+P`) to produce a PDF.

C. Role Extension for Export Endpoints

Initially, the export endpoints accepted only `admin` and `admin_dinas` roles. Extending to `super_admin` required adjusting both the role guard and the department-filter logic (Listing 3).

Listing 3. Export role guard and dept_filter logic

```
ALLOWED = ("admin", "admin_dinas", "super_admin")
if current_user.role not in ALLOWED:
    raise HTTPException(403, "Forbidden")

# dept_filter selection by role
if current_user.role == "admin_dinas":
    dept_filter = current_user.dept_id  # own dept
elif dept_id is not None:
    dept_filter = dept_id  # explicit query param
else:
    dept_filter = None  # super_admin: all depts
```

D. Citizen Feedback Document Viewer

When `admin_dinas` uploads a feedback document (`foto_feedback`), the filename is persisted in the `reports` table. Citizen-facing report cards conditionally render a *"View Document"* anchor element when `report.foto_feedback` is non-null, opening the file in a new browser tab and thereby providing full transparency of governmental action.

E. Admin Dashboard Filters

The admin and super-admin dashboards expose server-side filtering via query parameters (`status`, `start_date`, `end_date`). The React frontend binds these to:

- **Status pills:** Toggle buttons for `all|diterima|diproses|selesai|ditolak`.
- **Period selector:** Dropdown for Today / This Week / This Month / This Year / Custom Range.
- **Export bar:** CSV and PDF buttons sharing the active filter state.

State changes trigger a re-fetch via React `useEffect`, ensuring the table always reflects server-side data.

V. Results and Evaluation

A. Functional Testing

All 38 backend tests defined in `pytest` pass successfully, covering authentication, report CRUD, RBAC enforcement, AI routing, and export endpoints.

B. RBAC Enforcement

Table II summarizes access control test results for the most security-critical endpoints.

TABLE II
RBAC Enforcement Test Results

Endpoint	Role	Exp. / Act.
`GET /export/html`	`warga`	403 / 403 ✓
`GET /export/html`	`admin_dinas`	200 / 200 ✓
`GET /export/html`	`super_admin`	200 / 200 ✓
`POST /upload-feedback`	`warga`	403 / 403 ✓
`POST /upload-feedback`	`admin_dinas`	200 / 200 ✓
`PATCH /status`	`warga`	403 / 403 ✓

C. AI Routing Accuracy

The routing engine was evaluated on 50 manually crafted complaint descriptions spanning all registered departments. The system achieved a correct-department assignment rate of **88 %** and a correct urgency classification rate of **92 %**. Misclassifications were concentrated in cross-domain complaints (e.g., flooding near a school, which could route to either Public Works or Education).

D. Export Functionality

Prior to implementing the Blob Object URL pattern, export requests from `super_admin` sessions consistently returned HTTP 403. After the fix, all three admin-tier roles successfully download exports, with response times under 800 ms for up to 500 reports.

VI. Discussion

A. Scalability

The current implementation uses a synchronous SQLAlchemy session per request. For high-concurrency deployments, migrating to `asyncpg` with SQLAlchemy async sessions would improve throughput. Horizontal scaling is straightforward given the stateless JWT authentication model.

B. Limitations

1) The routing engine uses keyword matching; a fine-tuned transformer model (e.g., IndoBERT) would likely improve accuracy for ambiguous complaints.
2) There is no push-notification mechanism; citizens must actively refresh their dashboard.
3) File uploads are stored on the local filesystem, which does not support multi-instance deployments without a shared volume or object storage (e.g., MinIO, AWS S3).

C. Future Work

Planned enhancements include: (1) real-time notifications via WebSocket; (2) GIS heatmap analytics; (3) SLA tracking with automated escalation; and (4) an IndoBERT-based routing microservice.

VII. Conclusion

This paper presented **GovSmart**, a production-ready citizen complaint management system built on a modern React + FastAPI + PostgreSQL stack. The system addresses key gaps in existing e-government platforms: fine-grained RBAC, AI-assisted complaint routing, secure token-authenticated document export, and a citizen-facing feedback document viewer. All 38 automated tests pass, RBAC enforcement is verified across all role boundaries, and the AI routing engine achieves 88 % department accuracy. GovSmart demonstrates that a carefully engineered, relatively lean technology stack can deliver a robust, transparent, and auditable digital governance platform suitable for real-world municipal deployment.

Acknowledgment

The authors thank the faculty of the Department of Informatics Engineering, PENS, for guidance and infrastructure support throughout this project.

References

[1] B. W. Wirtz and P. Daiser, "E-government: Strategy, Process, and Instruments," *German University of Administrative Sciences Speyer*, 2019.

[2] A. Meijer, "E-governance innovation: Barriers and strategies," *Government Information Quarterly*, vol. 32, no. 2, pp. 198–206, 2015.

[3] O. Al-Hujran, M. Al-Debei, A. Chatfield, and M. Migdadi, "The imperative of influencing citizen attitude toward e-government adoption and use," *Computers in Human Behavior*, vol. 53, pp. 189–203, 2015.

[4] T. Janowski, "Digital government evolution: From transformation to contextualization," *Government Information Quarterly*, vol. 32, no. 3, pp. 221–236, 2015.

[5] Y. K. Dwivedi et al., "Re-examining the unified theory of acceptance and use of technology (UTAUT): Towards a revised theoretical model," *Information Systems Frontiers*, vol. 21, pp. 719–734, 2019.

[6] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, 2015. [Online]. Available: https://tools.ietf.org/html/rfc7519

[9] A. Bos, "URL.createObjectURL() — Web APIs," *MDN Web Docs, Mozilla*, 2022. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/URL/createObjectURL

[10] C. C. Aggarwal and C. Zhai, *Mining Text Data*. Springer, 2012.